

AGENTIC JOB HUNTING ENGINE

Implementation Blueprint, Architecture, Source Strategy & Runtime Design

Master specification for a user-configurable, multi-source job discovery pipeline

This document consolidates the requirements developed in the conversation and turns them into a reusable implementation plan. It deliberately separates JobSpy from the broader source ecosystem rather than pretending one scraper can cover the entire internet.

1. Core Requirement

When the launcher is run, the system must ask the user what kind of job they are hunting for. The scraper must not be permanently hard-coded to Product/UX/UI. The entered job target becomes the search profile for that run.

```
run_job_hunt.bat
> Enter target role family: Product Designer
> Optional seniority: Senior / Staff / Lead / Any
> Optional keywords: SaaS, Figma, B2B
> Start search? Y
```

A reusable search profile should be created from the input and passed to every source adapter. The system should still have a library of known role-expansion rules, exclusions, geography profiles, freshness rules and scoring signals, but the user controls the primary job family per run.

2. User-Configurable Search Input

- Required: target job or job family, e.g. Product Designer, Data Scientist, Frontend Engineer, Film Editor.
- Optional: seniority, e.g. Any, Senior, Staff, Lead, Principal, Head.
- Optional: preferred skills/keywords, e.g. Figma, React, Python, SaaS.
- Optional: excluded keywords, e.g. graphic design, unpaid, commission-only.
- Optional: employment type: full-time, contract, freelance, internship.
- Optional: geography preset: India + Middle East + Global, India only, Global only, custom.
- Optional: freshness override, default 72 hours with 0-24 and 24-72 buckets.

The run should save a machine-readable copy of the user search profile next to the outputs so the results can be reproduced or audited later.

3. Geography Model

India

- India nationwide
- Delhi
- Noida
- Gurgaon
- Bengaluru / Bangalore
- Pune

- Mumbai
- Chandigarh
- Hyderabad
- Mangalore / Mangaluru
- Kochi

Middle East

- Dubai, UAE
- Abu Dhabi, UAE
- Riyadh, Saudi Arabia
- Jeddah, Saudi Arabia
- Doha, Qatar
- Kuwait City, Kuwait
- Manama, Bahrain
- Muscat, Oman

Global

A separate Global / Rest of World region searches worldwide rather than relying on India or Middle East location values. The Global region is tracked independently in CSV/Excel and receives a guaranteed search budget.

4. Freshness Model

- Canonical freshness window: last 72 hours.
- Bucket A: 0-24 hours.
- Bucket B: 24-72 hours.
- Discard jobs older than 72 hours for the fresh-job datasets.
- Store source date, normalized date, age_hours, and scraped_at.
- Do not treat a generic "updated" timestamp as equivalent to posted date.

JobSpy exposes a hours_old parameter, but source behavior differs. Therefore hours_old should be used to reduce retrieval volume, while the pipeline performs its own final date/age calculation before classification.

5. Source Architecture

JobSpy is an ingestion layer. It is not the complete job universe. The system should use adapters with a shared normalized schema.

Layer	Examples	Integration method
JobSpy aggregators	LinkedIn, Indeed, Glassdoor, Google, Bayt, Naukri, ZipRecruiter	JobSpy adapter
Startup / specialist	Wellfound, Cutshort, Instahyre, Foundit, Dribbble, Behance, Coroflot	Dedicated adapters / compliant search endpoints
Remote / global	Remote OK, We Work Remotely, Himalayas, Remotive, Working Nomads, Arc	Dedicated adapters / feeds where available
Recruitment agencies	Michael Page, Randstad, Hays,	Direct search adapters

	Adecco, Robert Walters, Korn Ferry	
ATS / company careers	Greenhouse, Lever, Ashby, Workday, SmartRecruiters, Teamtailor, Personio, Pinpoint	ATS-specific adapters / public posting interfaces

6. Current JobSpy Reality

- The installed package in this project has been treated as the 1.1.82 line.
- Relevant documented backends for this geography include LinkedIn, Indeed, Glassdoor, Google, Bayt and Naukri; ZipRecruiter is primarily US/Canada-focused and should not consume India/Middle East budget.
- Naukri may return recaptcha_required; do not attempt to defeat CAPTCHA.
- Glassdoor can fail with location/parser errors; isolate the failure and continue.
- LinkedIn can rate-limit; pacing and bounded search budgets are required.
- Bayt location behavior is not equivalent to LinkedIn/Indeed, so its results need source metadata and downstream location validation.

The rule is: a source failure is a source-health event, not a pipeline failure.

7. Why the Old Search Strategy Failed

The earlier city-first loop exhausted one city before advancing. With dozens of search terms and multiple sources, the number of theoretical calls became several thousand. A 500-call limit therefore caused starvation: Delhi could consume the budget while Global was never reached.

```
BAD
for city:
  for term:
    for site:
      scrape()
```

```
BETTER
for region in scheduled_regions:
  rotate geography
  rotate query family
  rotate sources
```

8. Balanced Budget Scheduler

Retain hard safety limits, but make them a guardrail rather than the scheduling strategy. Initial allocation for a 500-search test run: India 45% (225), Middle East 30% (150), Global 25% (125).

The scheduler should interleave region queues rather than exhaust one region: India -> Middle East -> Global -> India -> Middle East -> Global. Within each region it should rotate locations. This guarantees that Noida/Gurgaon/Dubai/Global cannot be starved by Delhi.

```
MAX_SEARCHES = 500
MAX_RUNTIME_MINUTES = 90
INDIA_SHARE = 0.45
MIDDLE_EAST_SHARE = 0.30
GLOBAL_SHARE = 0.25
```

A future run can increase MAX_SEARCHES after observing source health. The limit should never be removed entirely.

9. Query Expansion

The user enters one primary target, and the engine derives related search phrases. For example, Product Designer can expand into Senior Product Designer, Staff Product Designer, Lead Product Designer, Principal Product Designer, Founding Product Designer, Product Design Lead, UX Designer, UI/UX Designer, Interaction Designer, Experience Designer, Design Systems Designer, UX Researcher, and similar terms. A different user-entered family would generate a different expansion dictionary.

Expansion should be role-family-specific and configurable in a YAML/JSON file, not hard-coded into main.py.

10. Relevance Classification

Use positive and negative signals rather than relying entirely on search-engine keyword matching. The classifier should prioritize job-title evidence, then use description evidence for refinement.

- Positive role/title terms increase confidence.
- Clearly unrelated categories become negative title signals.
- The system stores classification reason, not just a boolean.
- Classifiers should be replaceable without changing source adapters.

11. Scoring

The score is a prioritization mechanism, not proof of suitability.

- Role fit: strongest weight.
- Seniority: senior/staff/lead/principal/founding/head/director signals.
- Skill overlap: target skills and technologies.
- Freshness: extra points for 0-24 hours.
- Visa sponsorship: positive signal for international roles.
- Relocation: positive signal.
- Explicit negative work-authorization constraints: penalty where relevant.

12. Visa / Relocation Intelligence

Every international result should receive Sponsorship and Relocation fields. UNKNOWN is not equivalent to NO.

- SPONSORSHIP MENTIONED
- NO SPONSORSHIP
- UNKNOWN
- RELOCATION MENTIONED
- UNKNOWN

The next implementation should additionally store `visa_evidence` and `relocation_evidence`: the exact short phrase that triggered the classification. This makes the automation auditable.

13. ATS Strategy

ATS integration is one of the highest-value expansions because companies can publish directly through recruiting systems without relying on a social job board. The first ATS implementations should target Greenhouse, Lever, Ashby and SmartRecruiters; Workday and other enterprise career systems follow as dedicated adapters.

Ashby currently documents a public Job Posting API that can return published job postings, including location and optional compensation. SmartRecruiters documents public Posting API endpoints for active company postings with query and location filters. These interfaces make ATS adapters materially cleaner than brittle page scraping.

14. Adapter Contract

```
class JobSourceAdapter:
    name: str
    regions: list[str]

    def search(self, query, location, freshness_hours) -> list[RawJob]:
        ...

    def health(self) -> SourceHealth: ...
```

Every adapter returns the same `RawJob` contract. The core engine never needs to know whether a record came from `JobSpy`, an ATS API, a recruiter site, or a specialist board.

15. Normalized Job Schema

- `job_id / source_job_id`
- `source`
- `source_url`
- `job_url`
- `company`
- `title`
- `location`
- `country`
- `region`
- `description`
- `date_posted`
- `scraped_at`
- `age_hours`
- `employment_type`
- `remote_status`
- `salary_min / salary_max / currency`
- `visa_status`
- `visa_evidence`

- relocation_status
- relocation_evidence
- match_score
- priority
- query_used
- search_location

16. Deduplication

The same vacancy can appear on LinkedIn, Indeed, an ATS and a recruiter page. Deduplicate in stages: exact URL, normalized title+company+location, then stronger similarity matching when identifiers are available. Preserve a sources array so one canonical job can show all places it was discovered.

17. Run Outputs

Every run must create new timestamped files. No run should overwrite another run.

- JOBSPY_ALL_72H_<timestamp>.csv
- JOBSPY_LAST_24H_<timestamp>.csv
- JOBSPY_24_TO_72H_<timestamp>.csv
- JOBSPY_INDIA_<timestamp>.csv
- JOBSPY_MIDDLE_EAST_<timestamp>.csv
- JOBSPY_GLOBAL_REST_OF_WORLD_<timestamp>.csv
- JOBSPY_VISA_SPONSORSHIP_<timestamp>.csv
- JOBSPY_RAW_<timestamp>.csv
- JOBSPY_RUN_SUMMARY_<timestamp>.csv
- JOBSPY_RUN_SUMMARY_<timestamp>.txt
- JOBSPY_RUN_<timestamp>.xlsx
- JOBSPY_SEARCH_PROFILE_<timestamp>.json

18. Excel Workbook

- RAW_RESULTS
- ALL_72H
- LAST_24H
- 24_TO_72H
- INDIA
- MIDDLE_EAST
- GLOBAL
- VISA_SPONSORSHIP
- SOURCE_HEALTH
- RUN_SUMMARY

19. Runtime & Exit Strategy

- Normal completion: all scheduled searches finish; process exits.

- MAX_SEARCHES reached: stop before starting another source call.
- MAX_RUNTIME_MINUTES reached: stop before starting another source call.
- Ctrl+C: immediate manual stop; preserve any already-written checkpoint/output files.
- Source error: log and continue, never terminate the entire run.

For long runs, write a checkpoint after each completed search or batch. That makes interruption recoverable rather than forcing a full restart.

20. Launcher / BAT Design

The BAT launcher should no longer be a fixed Product Design runner. It should invoke a Python CLI that asks for the job target.

```
@echo off
cd /d C:\JobSpy
python main.py
cmd /k
```

The Python CLI asks for the target role and other optional filters, then constructs the search profile. The terminal remains open after the run so errors can be inspected.

21. Project Structure

```
C:\JobSpy\
main.py
run_job_hunt.bat
config\
  roles.yaml
  locations.yaml
  sources.yaml
core\
  scheduler.py
  normalizer.py
  deduper.py
  classifier.py
  scorer.py
  visa.py
  freshness.py
  exporters.py
  checkpoint.py
sources\
  jobspy_adapter.py
  wellfound.py
  greenhouse.py
  lever.py
  ashby.py
  smartrecruiters.py
  workday.py
  recruiter_sites.py
  design_boards.py
  remote_boards.py
output\
logs\
checkpoints\
```

22. Development Phases

Phase 1 - Stable JobSpy core: User-configurable query; India/Middle East/Global; 72h freshness; balanced budgets; timestamped outputs; source isolation.

Phase 2 - High-value specialist sources: Wellfound, Cutshort, Instahyre, Foundit, Dribbble, Behance, Coroflot, selected remote boards.

Phase 3 - ATS engine: Greenhouse, Lever, Ashby, SmartRecruiters, Workday, Teamtailor, Personio, Pinpoint and direct company career pages.

Phase 4 - Recruitment agencies: Michael Page, Randstad, Hays, Adecco, Robert Walters, Korn Ferry and high-value regional recruiters.

Phase 5 - Agentic workflow: Open pages, extract structured fields, rank, generate tailored application material, write to tracking database, and schedule follow-up reminders.

23. Agentic Job Application Layer

The broader goal is not just collecting jobs. Once discovery is stable, the system can become an agentic application pipeline: discover -> qualify -> open source page -> extract requirements -> tailor resume/cover letter -> prepare application -> log outcome -> remind on follow-up. Application submission must remain compliant with site access rules and should not bypass CAPTCHA, authentication controls or anti-bot protections.

24. Source Compliance & Reliability

- Prefer official/public APIs, feeds and ATS posting interfaces where available.
- Respect terms, robots.txt, authentication requirements, rate limits and access controls.
- Never defeat CAPTCHA or anti-bot challenges.
- Store source health and error reasons.
- Use retries with backoff only for transient errors.
- Keep adapters independent so one source cannot block the run.

25. Acceptance Tests

- Run asks the user for target job type before scraping.
- Changing target from Product Designer to Data Analyst changes the search profile without code edits.
- Delhi cannot consume all India search budget before Noida/Gurgaon get turns.
- Middle East and Global receive guaranteed allocations.
- A Naukri CAPTCHA does not terminate the run.
- A Glassdoor parser/location failure does not terminate the run.
- No result older than 72 hours appears in the fresh datasets.
- Last-24h and 24-72h datasets are separate and correctly timestamped.
- A separate Global output exists for every run.
- Every run produces a summary with start/end time, stop reason, search count and result counts.
- Duplicate jobs across sources collapse into one canonical job while preserving source provenance.
- Ctrl+C stops cleanly.

26. Research Notes / Current Source Evidence

Current official documentation confirms that Ashby exposes a public Job Posting API for published postings and SmartRecruiters documents public Posting API endpoints for active company postings. These are strong candidates for ATS adapters. Wellfound publishes separate Jobs Terms governing access to its job service, so the integration should be treated as a source-specific, terms-aware adapter rather than assumed to be an unrestricted API.

The broader source list is a prioritization set, not a claim that every named platform currently provides a public scraping API. Each adapter should be verified against the platform at implementation time.

27. Recommended Initial Configuration

```
MAX_SEARCHES = 500
MAX_RUNTIME_MINUTES = 90
MAX_HOURS = 72
INDIA_SHARE = 0.45
MIDDLE_EAST_SHARE = 0.30
GLOBAL_SHARE = 0.25
RESULTS_PER_SITE = 25
REQUEST_DELAY = 2.5
```

This is a controlled test configuration. After observing coverage and source health, the budget can be increased. The scheduler must retain regional guarantees even as the total budget changes.

28. End State

```
USER QUERY
-> SEARCH PROFILE
-> REGION/SOURCE SCHEDULER
-> SOURCE ADAPTERS
-> NORMALIZATION
-> DEDUPLICATION
-> ROLE CLASSIFICATION
-> FRESHNESS
-> VISA / RELOCATION
-> MATCH SCORE
-> INDIA / MIDDLE EAST / GLOBAL
-> CSV / EXCEL / LOG / DATABASE
-> AGENTIC APPLICATION WORKFLOW
```

The system should behave like a personal job-search engine with a modular ingestion layer, not a single scraper tied to one profession or one website.

Sources

JobSpy repository / documentation: <https://github.com/speedyapply/JobSpy>

Ashby Public Job Posting API: <https://developers.ashbyhq.com/docs/public-job-posting-api>

SmartRecruiters Posting API: <https://developers.smartrecruiters.com/docs/posting-api>

Wellfound Jobs Terms: <https://wellfound.com/terms/jobs>